

Technical Reference

SECTION 1 — SQL QUERIES

1.1 Database Setup & Schema

```
CREATE DATABASE bank_churn;

USE bank_churn;

-- Set primary key after import
ALTER TABLE bank_churn
MODIFY CustomerId BIGINT PRIMARY KEY;

-- Verify schema
DESC bank_churn;
```

1.2 Exploratory Data Analysis

```
-- Row count
SELECT COUNT(*) FROM bank_churn;

-- Null check
SELECT SUM(ISNULL(CreditScore)), SUM(ISNULL(Age)),
       SUM(ISNULL(Balance)), SUM(ISNULL(Geography)),
       SUM(ISNULL(Gender)) FROM bank_churn;

-- Duplicate check
SELECT CustomerId, COUNT(*) AS cnt
FROM bank_churn GROUP BY CustomerId HAVING cnt > 1;

-- Basic stats
SELECT ROUND(AVG(Age),1) AS avg_age,
       ROUND(AVG(CreditScore),1) AS avg_credit_score,
       ROUND(AVG(Balance),2) AS avg_balance,
       ROUND(AVG(EstimatedSalary),2) AS avg_salary,
       ROUND(AVG(Tenure),1) AS avg_tenure
FROM bank_churn;
```

1.3 Churn Analysis Queries

```
-- Overall churn rate
SELECT COUNT(*) AS total_customers,
       SUM(Exited) AS churned,
       ROUND(SUM(Exited)*100.0/COUNT(*),2) AS churn_rate_pct
FROM bank_churn;

-- Churn by Geography
SELECT Geography, COUNT(*) AS total, SUM(Exited) AS churned,
       ROUND(SUM(Exited)*100.0/COUNT(*),2) AS churn_rate_pct
FROM bank_churn GROUP BY Geography ORDER BY churn_rate_pct DESC;
```

```

-- Churn by Gender

SELECT Gender, COUNT(*) AS total, SUM(Exited) AS churned,
       ROUND(SUM(Exited)*100.0/COUNT(*),2) AS churn_rate_pct
FROM bank_churn GROUP BY Gender ORDER BY churn_rate_pct DESC;

-- Churn by Age Group

SELECT CASE WHEN Age < 30 THEN 'Under 30'
           WHEN Age BETWEEN 30 AND 40 THEN '30-40'
           WHEN Age BETWEEN 41 AND 50 THEN '41-50'
           WHEN Age BETWEEN 51 AND 60 THEN '51-60'
           ELSE 'Above 60' END AS age_group,
       COUNT(*) AS total, SUM(Exited) AS churned,
       ROUND(SUM(Exited)*100.0/COUNT(*),2) AS churn_rate_pct
FROM bank_churn GROUP BY age_group ORDER BY churn_rate_pct DESC;

-- Churn by Number of Products

SELECT NumOfProducts, COUNT(*) AS total, SUM(Exited) AS churned,
       ROUND(SUM(Exited)*100.0/COUNT(*),2) AS churn_rate_pct
FROM bank_churn GROUP BY NumOfProducts ORDER BY NumOfProducts;

```

1.4 Behavioral & Financial Queries

```

-- Churn by Active Member status

SELECT IsActiveMember, COUNT(*) AS total, SUM(Exited) AS churned,
       ROUND(SUM(Exited)*100.0/COUNT(*),2) AS churn_rate_pct
FROM bank_churn GROUP BY IsActiveMember;

-- Churn by Credit Score bucket

SELECT CASE WHEN CreditScore < 400 THEN 'Poor (<400)'
           WHEN CreditScore BETWEEN 400 AND 579 THEN 'Fair (400-579)'
           WHEN CreditScore BETWEEN 580 AND 669 THEN 'Good (580-669)'
           WHEN CreditScore BETWEEN 670 AND 739 THEN 'Very Good (670-739)'
           ELSE 'Excellent (740+)' END AS credit_bucket,
       COUNT(*) AS total, SUM(Exited) AS churned,
       ROUND(SUM(Exited)*100.0/COUNT(*),2) AS churn_rate_pct
FROM bank_churn GROUP BY credit_bucket ORDER BY churn_rate_pct DESC;

-- Churn by Balance bucket

SELECT CASE WHEN Balance = 0 THEN 'Zero Balance'
           WHEN Balance < 50000 THEN 'Low (<50k)'
           WHEN Balance BETWEEN 50000 AND 100000 THEN 'Mid (50k-100k)'
           WHEN Balance BETWEEN 100000 AND 150000 THEN 'High (100k-150k)'
           ELSE 'Very High (150k+)' END AS balance_bucket,
       COUNT(*) AS total, SUM(Exited) AS churned,
       ROUND(SUM(Exited)*100.0/COUNT(*),2) AS churn_rate_pct
FROM bank_churn GROUP BY balance_bucket ORDER BY churn_rate_pct DESC;

-- High value customers already lost

SELECT COUNT(*) AS high_value_at_risk

```

```
FROM bank_churn WHERE Balance > 100000 AND Exited = 1;
```

1.5 Customer Segmentation Queries

```
-- High risk segment (not yet churned)

SELECT COUNT(*) AS high_risk FROM bank_churn
WHERE IsActiveMember = 0 AND Age BETWEEN 41 AND 60
AND Geography = 'Germany' AND Exited = 0;

-- Most loyal segment

SELECT COUNT(*) AS most_loyal FROM bank_churn
WHERE IsActiveMember = 1 AND Age < 30
AND NumOfProducts = 2 AND Exited = 0;

-- High value retained by geography and gender

SELECT Geography, Gender,
       ROUND(AVG(Balance),2) AS avg_balance,
       ROUND(AVG(EstimatedSalary),2) AS avg_salary,
       COUNT(*) AS total
FROM bank_churn WHERE Exited = 0 AND Balance > 100000
GROUP BY Geography, Gender ORDER BY avg_balance DESC;

-- Churned vs retained comparison

SELECT Exited, ROUND(AVG(Age),1) AS avg_age,
       ROUND(AVG(CreditScore),1) AS avg_creditscore,
       ROUND(AVG(Balance),2) AS avg_balance,
       ROUND(AVG(Tenure),1) AS avg_tenure,
       ROUND(AVG(NumOfProducts),2) AS avg_products
FROM bank_churn GROUP BY Exited;
```

SECTION 2 — DAX MEASURES

2.1 KPI Measures

```
-- Total Customers
Total Customers = COUNTROWS(bank_churn)

-- Churn Rate %
Churn Rate % =
DIVIDE(
    COUNTROWS(FILTER(bank_churn, bank_churn[Exited] = 1)),
    COUNTROWS(bank_churn), 0) * 100

-- Active %
Active % =
DIVIDE(
    COUNTROWS(FILTER(bank_churn, bank_churn[IsActiveMember] = 1)),
    COUNTROWS(bank_churn), 0) * 100

-- High Value Lost
High Value Lost =
```

```
COUNTROWS(FILTER(bank_churn,
    bank_churn[Exited] = 1 && bank_churn[Balance] > 100000))
```

2.2 Profile Measures

```
Avg Age = AVERAGE(bank_churn[Age])

Avg Balance = AVERAGE(bank_churn[Balance])

Avg Tenure = AVERAGE(bank_churn[Tenure])
```

2.3 Calculated Columns

```
-- Age Group
Age Group =
SWITCH(TRUE(),
    bank_churn[Age] < 30, 'Under 30',
    bank_churn[Age] <= 40, '30-40',
    bank_churn[Age] <= 50, '41-50',
    bank_churn[Age] <= 60, '51-60',
    'Above 60')

-- Estimated Income Bucket
Estimated Income =
SWITCH(TRUE(),
    bank_churn[EstimatedSalary] < 50000, 'Low (<50k)',
    bank_churn[EstimatedSalary] < 100000, 'Mid (50k-100k)',
    bank_churn[EstimatedSalary] < 150000, 'High (100k-150k)',
    'Very High (150k+)')
```

SECTION 3 — POWER BI FEATURES USED

Get Data → MySQL Database	Connected Power BI directly to MySQL Workbench via MySQL Connector/NET. Server: localhost, Database: bank_churn.
Card Visual	Used for all 4 KPI tiles: Total Customers, Churn Rate%, Active%, High Value Lost. Format → Callout value → Display units set to None for exact values.
Clustered Bar Chart	Used for Churn by Geography, Churn by Age Group, Churn by Active Member, Churn by Estimated Income.
Clustered Column Chart	Used for Churn by Number of Products — column chart chosen to show the product paradox escalation clearly.
Donut Chart	Used for Churn by Gender — shows Male vs Female split with percentages.
Matrix Visual	Used for Churned vs Retained Profile table — Exited in Rows, Avg Age / Avg Tenure / Avg Balance in Values.
Numeric Range Slicer	Used for Credit Score filter (350–850 range) — allows dashboard filtering by credit score band.

Measure	Used to create all DAX KPI and profile measures (Churn Rate%, Active%, High Value Lost, Avg Age, Avg Balance, Avg Tenure).
Clustered Bar Chart	Used to create calculated columns: Age Group and Estimated Income — both use SWITCH(TRUE()) pattern for bucketing.